EDUCATOR ANALYTICS OS

Complete Project Document

Architecture · Technology · Features · Requirements · Decisions

University Project | Multi-Agent AI Platform | LangGraph + Claude API

CSE Department · 2026

Table of Contents

PART 1

Project Vision & Overview

What We Are Building

Educator Analytics OS is a multi-role, AI-powered student intelligence platform designed for universities and engineering colleges. It is not a dashboard that shows data. It is a living system that collects every signal from every student — marks, attendance, projects, extracurriculars, sports, communication skills — processes it through a network of specialized AI agents built with LangGraph and Claude, and delivers the right insight to the right person at the right time, automatically.

The platform serves five types of users simultaneously: students who want to understand their own potential and career path, faculty who want to identify struggling students before exams, deans who want to make data-driven department decisions, parents who want to understand their child's real strengths and weaknesses, and administrators who need accreditation-ready reports at the click of a button.

> *The core belief this project is built on: every student is more than their marks. A student who scores 50% in theory exams but 90% in project work is not a weak student — they are a builder who learns differently. This platform sees that distinction and communicates it to everyone who interacts with that student.*

The Problem It Solves

Indian engineering colleges today operate with fragmented systems. Attendance is tracked in one portal. Marks are entered in another spreadsheet. Project submissions happen over email. Sports and extracurricular achievements are noted in a register no one reads. A parent visiting college talks to a faculty member who has to pull data from four different places and still cannot give a complete picture of their child.

The dean makes curriculum decisions based on gut feeling because there is no tool that shows which subjects are consistently failing across batches. A student approaching placement season has no way to know exactly which skills they lack for their target company. A faculty member discovers a student is at risk only after they fail the semester exam — too late to intervene.

Educator Analytics OS solves all of this by being the single nervous system of the institution — one platform where every signal flows in, gets intelligently processed, and the right action gets surfaced to the right person before problems become permanent.

The Scale of the Vision

This is not built to serve one college. The architecture is designed for multi-tenancy from day one — meaning one deployment can serve hundreds of institutions, each seeing only their own data with their own configuration. When a second college onboards, they configure their branch names, grading system, and semester structure, and the entire platform adapts. This is the architecture of a SaaS product, not a college project.

Multi-Tenant SaaS	**6 AI Agents**	**5 User Roles**	**Real-Time Insights**

PART 2

How This Changes the Department

Before This Platform Exists

A typical engineering department today operates reactively. Problems are discovered after they become serious. A student who was struggling for three months gets flagged only when they fail a unit test. A batch with poor placement outcomes reveals a curriculum gap only after the placement season ends. Faculty spend hours compiling reports manually that could be generated in seconds. Parents are informed of their child's condition only during parent-teacher meetings scheduled twice a year.

Area	Before the Platform	After the Platform
Attendance tracking	Manual registers or basic ERP, percentage only	Day-level, subject-level, auto-alerts when threshold crossed
Student performance	Semester-end marks in spreadsheets	Real-time, assessment-level, trend analysis, AI-flagged risks
Parent communication	Twice-yearly meetings, informal calls	Weekly WhatsApp digest, instant alerts, parent visit mode
Career guidance	Faculty advice based on experience	AI-driven career path with skill gap analysis and alumni data
Accreditation reports	Weeks of manual compilation	One-click NBA/NAAC report generation from live data
Curriculum decisions	Based on faculty experience and gut	Data-driven: subject-wise failure analysis, industry alignment
Student strengths	Only academic marks considered	7-dimension profile: academics, projects, sports, communication
At-risk detection	After exam failure — too late	4-6 weeks before exams, automatic intervention recommendations

The Transformation in Daily Operations

On the day a faculty member opens the platform each morning, they see not a blank dashboard but a prioritized view of what needs their attention today. Which students are showing decline. Which class has attendance below threshold. Which assignment deadline passed with low submission rates. The platform has already done the analysis overnight. The faculty member's job shifts from data gatherer to action taker.

For the dean, the weekly department review changes completely. Instead of asking faculty to prepare reports, the dean opens a single screen showing department health scores for every branch, placement readiness trends for final year students, and curriculum recommendations generated by AI from analyzing three years of student performance data. Strategic decisions that previously required weeks of data gathering now take minutes.

For students, the experience of understanding their own potential changes. Instead of looking at a marksheet and feeling uncertain about their future, a student opens their profile and sees a clear picture: their strongest skill dimensions, the exact gap between their current skills and their target career, a 30-day action plan to close that gap, and examples of alumni who had a similar profile and where they ended up. This replaces anxiety with direction.

> *The department that uses this platform for three years will have three years of structured student intelligence data that no competitor, no other institution, and no vendor can replicate. That data becomes their most valuable academic asset.*

PART 3

Stakeholder Relationships & Roles

The platform is built around five distinct roles, each with their own view, their own data access, their own AI interactions, and their own use cases. Understanding how these roles connect is essential to understanding the platform.

Faculty

Faculty are the primary daily users of the platform. They interact with it multiple times every day — marking attendance, entering assessment scores, reviewing student alerts, and communicating with students and parents. The platform is designed to make their existing job significantly easier while also giving them intelligence they never had access to before.

What faculty can do:

Feature	Description
Subject analytics	View a complete topic-wise score heatmap across the entire class. See exactly which concepts the class understood and which they failed, broken down to the individual topic level within each subject. The system identifies the five students who need the most support in each topic and surfaces them automatically so faculty can prioritize their intervention effort where it matters most.
Student profiles	Open any student's complete living profile from a single search. The profile shows the full academic record across all subjects and semesters, project submissions with AI quality scores, extracurricular achievements, faculty behavioral notes, skill radar chart, AI-generated career predictions, alert history, and placement readiness score — everything in one place without navigating multiple systems.
Alert management	Receive AI-generated alerts ranked by severity for every student in the faculty's subjects. Each alert includes the specific data points that triggered it, a recommended intervention action based on what has worked for similar cases historically, and a follow-up tracker so the faculty can log what action they took and the system can track whether it led to improvement over the next 30 days.

Feature	Description
Parent communication	Send direct messages to parents of any student through the platform with full message history. Generate a one-tap parent visit report that summarizes the student's complete academic standing, strengths, concerns, and recommended parental actions in simple language. Schedule and manage parent-teacher meetings with automated reminders sent to both parties.
CO attainment tracking	Track the attainment percentage of every Course Outcome mapped to the faculty's subjects in real time, updated automatically as assessment data flows in from the connected ERP. See at a glance which outcomes are on track, which are at risk, and which have been achieved — with the detailed assessment-level breakdown available on demand for accreditation purposes.
Report generation	Generate comprehensive class progress reports, individual student narrative reports, and full section-level analytics reports with a single click. Every report is formatted professionally, includes AI-generated commentary on trends and patterns, and can be exported as PDF or shared directly with the dean or parents through the platform.

Dean / Head of Department

The dean interacts with the platform at a strategic level. They are not looking at individual students daily — they are looking at the health of the entire department, trends across batches, faculty performance, curriculum gaps, and placement outcomes. The AI layer built specifically for the dean translates raw department data into actionable strategic insights.

What the dean can do:

Feature	Description
Department health dashboard	A single-screen strategic overview of every branch and every year in the department simultaneously. Shows real-time department health scores per branch, total active alert counts, placement readiness trend lines for final year students, SPI distribution across cohorts, and a faculty workload summary — giving the dean a complete picture of departmental health in seconds without opening a single sub-report.

Feature	Description
Faculty performance review	A data-driven view of how much students actually improve under each faculty member, measured by the change in student performance scores before and after each faculty's teaching period. This metric is based entirely on outcome data — not student satisfaction ratings — making it an objective and fair basis for faculty development conversations, recognition, and resource allocation decisions.
Curriculum gap analysis	Identifies which subjects, topics, and course outcomes are consistently failing across multiple batches and multiple years. The system surfaces patterns that no human reviewer could see manually — for example, identifying that normalization in DBMS has been the single most failed topic across three consecutive batches, providing concrete evidence for curriculum revision decisions rather than relying on faculty intuition.
Cohort forecasting	AI-generated predictions for the current batch updated on a rolling basis throughout the semester. Includes predicted placement rate at current trajectory, predicted dropout risk students by name with confidence percentages, expected CGPA distribution at semester end, and identification of which students are most likely to improve significantly with targeted intervention. Updated weekly as new assessment data flows in.
Policy simulation	An AI-powered scenario planning tool where the dean can ask what-if questions about curriculum or policy changes. For example: if a communication skills lab is added in the second semester of second year, what is the predicted impact on placement readiness scores based on the correlation between communication skills and placement outcomes in historical data? The system provides evidence-based projections to support strategic decisions.
Accreditation reports	One-click generation of complete NBA and NAAC accreditation reports compiled automatically from live institutional data. Every required metric — CO attainment rates, student performance analytics, outcome-based education evidence, faculty qualification records, and curriculum review documentation — is pulled from the database and formatted to match the exact specifications of the accreditation body. What previously required weeks of manual work is completed in minutes.

Feature	Description
Comparative batch analytics	Side-by-side comparison of the current batch's performance against previous batches at the exact same point in the academic calendar. Shows whether the current batch is trending ahead, behind, or aligned with historical norms — and identifies which specific subjects or skill areas show the greatest divergence, enabling early corrective action rather than reactive response at semester end.
Cross-branch insights	A unified view comparing performance, SPI distributions, placement readiness, and skill profiles across CSE, IT, ECE, and other branches simultaneously. Identifies which branches are excelling in which dimensions and which are underperforming, providing the dean with evidence-based data for resource allocation, faculty deployment, and inter-department collaboration decisions.

Student

The student view is designed to be motivating, not punishing. Every feature is built to answer one question: what should I do next to improve my future? The platform shows students not just where they are but where they could be, and gives them a concrete path to get there.

Feature	Description
Personal dashboard	A fully personalized home screen summarizing the student's current standing across every domain simultaneously — academic performance and SPI score, development and project activity, sports and extracurricular achievements, communication skill level, and upcoming deadlines. No domain is hidden or deprioritized. The student sees their complete self, not just their marks.
Smart notifications	A unified notification center that aggregates all alerts the student needs to act on — new assignment postings with deadlines, attendance warnings before the threshold is crossed, important announcements from faculty, AI-generated nudges when the 30-day action plan has a pending task, and positive milestone notifications when a skill level improves or a rank changes favorably.
Multi-domain rank	The student's rank is shown separately and transparently for each domain — academic rank within their section, academic rank within the entire branch, development and project rank among peers, communication and presentation rank, sports and extracurricular rank, and overall SPI rank. A student who ranks 40th in academics but 3rd in development sees both numbers clearly. Performance is never collapsed into a single misleading rank.
Branch domain directory	A searchable, filterable directory of every student in the branch organized by their strongest domain — Web Development, AI and Machine Learning, Data Science, IoT and Embedded Systems, UI and Design, Competitive Programming, Research, and more. Each entry shows the student's top skills, notable projects, and current availability for collaboration. This is the tool that makes finding the right teammate for a hackathon or capstone project take minutes instead of days.
Team formation assistant	A structured team-building workflow where a student can browse the domain directory, filter by specific skills needed for a project, view each potential teammate's project history and skill radar, and send a formal team invite with a project description directly through the platform. Manages team formation for hackathons, course projects, and competitions with invitation tracking and acceptance notifications.

Feature	Description
Teammate quick access	Once a team is formed, every team member gets a dedicated collaboration space showing all teammates' complete profiles — their skill radars, current academic workload, strongest technical domains, project history, and contact information. The space also tracks the team's shared project submissions and milestones, making it the single source of truth for every collaborative project.
7-dimension radar chart	A visual radar chart mapping the student's strength across all seven intelligence dimensions — Logical-Mathematical derived from exam and problem-solving performance, Linguistic from assignment and communication scores, Spatial-Creative from design and project work, Kinesthetic from lab and sports performance, Interpersonal from group project outcomes and leadership, Intrapersonal from consistency and discipline, and Technical Depth from code quality and technology exposure. Updated in real time as new data is recorded.
Placement readiness score	A dynamic 0-100 score calculated from the student's full profile — not just marks — showing how prepared they are for campus placement right now. The score updates as new assessments are entered, new skills are demonstrated, and new projects are submitted. Accompanied by a specific breakdown of the top three actions that would increase the score most significantly, with estimated impact percentages for each.
Career path recommendations	The Career Advisor Agent analyzes the student's complete 7-dimension profile, academic performance patterns, project portfolio, and extracurricular record to recommend the top three career paths that align with their actual demonstrated strengths. Each recommendation includes a confidence percentage, the specific profile characteristics that support the recommendation, and the skill gap between the student's current level and what is needed to excel in that path.
Skill gap to dream job	The student enters their target company, role, or career path and the system immediately shows the precise gap between their current skill levels and the requirements for that target. Each gap item comes with specific learning resources — curated courses, practice platforms, project ideas — prioritized by how much closing that gap would improve the student's chances of achieving their target.

Feature	Description
10-year trajectory simulator	Three parallel future scenarios built from the student's current data profile. The first scenario shows the likely outcome if the student continues at their current trajectory without significant changes. The second shows the outcome if they focus effort on improving in their top two skill gap areas. The third shows an alternative path if they pivot toward a different career domain that better matches their natural strengths. Each scenario includes projected role, approximate compensation range, and the specific actions that lead there.
Alumni mirror	A curated list of students from previous batches who had a similar skill profile, academic standing, and extracurricular pattern to the current student. Shows where each alumni is now — company, role, and career path — based on actual outcome data collected from the institution's alumni network. This transforms abstract career advice into concrete proof points from people who started in the same position.
30-day action plan	An AI-generated personalized monthly plan that identifies the three to five highest-impact actions the student can take in the next 30 days to improve their placement readiness score and career path alignment. Each action is specific — not 'improve your coding skills' but 'complete this specific SQL module, it addresses your weakest subject gap and has the highest correlation with placement success in your target domain.' The plan updates automatically at the start of each month based on progress.
Resume builder	An automatically generated professional resume built entirely from the student's actual platform data — skills verified through assessments and projects, work and project experience from the project portfolio, achievements from extracurricular records, certifications from the skills log, and a personal profile statement drafted by AI from the student's overall profile. The resume updates as the student's profile grows and can be downloaded as a formatted PDF at any time.

Parent

Parents are occasional users of the platform. They do not have a permanent login. Instead, they receive weekly WhatsApp digests and can access their child's information during physical college visits through a QR-based temporary access system generated by faculty. The parent view is designed to be simple, visual, non-technical, and available in Hindi and regional languages.

Feature	Description
Weekly WhatsApp digest	An automated weekly summary sent directly to the parent's WhatsApp number every Sunday — no app installation, no login, no technical knowledge required. The digest covers the week's attendance summary with any sessions missed, the most recent assessment scores if any were entered that week, the current SPI trend direction, and any active alerts that need parental attention. Written in simple, non-technical language and available in Hindi or the parent's preferred regional language.
Parent visit mode	When a parent visits the college, the faculty or dean generates a temporary QR code that expires after the meeting. The parent scans the QR code on any mobile browser and immediately sees a clean, visual, one-screen summary of their child's complete status — attendance health, academic standing, strongest skill dimensions, areas needing improvement, behavioral observations, AI-predicted outcomes, and specific recommended actions for the parent. No login account is ever needed.
Instant alerts	Real-time push notifications sent to the parent's mobile number the moment a significant event occurs — attendance falls below the 75% threshold, a major assessment score shows a significant drop from the student's personal baseline, three consecutive assignments are missed, or the student's SPI score changes significantly in either direction. Each alert includes a brief explanation of why it was sent and a suggested conversation the parent can have with their child.
Child's strength view	A dedicated section that surfaces what the student is genuinely good at — going far beyond marks. Shows the student's strongest skill dimensions from the radar chart, their best projects with faculty ratings, sports and extracurricular achievements, and any certifications completed. This view is specifically designed to ensure parents understand their child's full potential, not just their academic weaknesses, which is often the only information shared during traditional parent-teacher interactions.
Recommended actions	After every significant data change or parent visit, the AI generates a specific, personalized list of actions the parent can take to support their child's situation. These are not generic suggestions — they are derived from the student's specific profile. If a student is strong in projects but weak in theory exams, the recommended action might be to ensure a quiet study time for two weeks before unit exams rather than a generic 'study more' suggestion.

Feature	Description
Meeting scheduler	Parents can request a parent-teacher meeting directly through the platform or via a link in the WhatsApp digest. The system shows available time slots from the faculty's calendar, allows the parent to select a slot, and sends confirmed calendar invites with reminders to both parent and faculty 24 hours before the meeting. Meeting notes can be added by faculty after the meeting and are visible to the parent.
PDF summary download	A one-tap button in the parent visit mode that generates a complete, formatted PDF summary of the student's profile — academic performance, skill radar chart, attendance record, AI predictions, and recommended actions. The PDF is in the parent's preferred language, clearly laid out with visual elements, and designed to be printable and shareable with family members who want to understand the student's standing.

Administrator

The administrator manages the institutional configuration of the platform. They create faculty accounts, configure branch structures, set grading systems, manage SPI weight configurations, and handle system-wide settings. The admin role is what enables multi-tenancy — each institution has its own admin who configures the system without touching any other institution's data.

PART 4

Complete Feature Breakdown

Student Potential Index (SPI)

The Student Potential Index is the single most important feature of the platform. It is a 0-100 score that represents the complete picture of a student's potential — not just academic marks. It is the number that appears on every screen, every report, and every alert. It is the score a dean can glance at to understand a student's overall standing in 2 seconds.

The SPI is calculated from five weighted dimensions. Academic performance contributes 30% — this includes exam scores, assignment quality, quiz results, and CO attainment. Skill breadth contributes 20% — covering the range of technical and soft skills demonstrated. Project quality contributes 20% — scored by both faculty rating and AI analysis of submission complexity. Consistency and growth trajectory contributes 20% — a student improving steadily scores higher than one who peaked early and declined. Extracurricular and communication activities contribute the remaining 10%. The weights are configurable by each institution.

SPI is used by: Faculty (to prioritize daily attention), Dean (department health), Students (personal goal tracking), Parents (child's overall standing), Alert Agent (trigger threshold for risk detection).

Living Student DNA Profile

Every student has a single profile that aggregates all data points ever recorded about them. This profile updates in real time as new data is entered. When a faculty member enters marks after an exam, the profile updates within seconds. When a student submits a project, the AI scores it and the profile reflects it immediately. The profile is structured into clear sections: Academic Health, Skill Radar, Project Portfolio, Extracurricular Timeline, Alert History, Behavioral Notes, AI Predictions, and Placement Readiness.

The profile is the foundation of the parent visit mode. When a parent walks into college, the faculty opens this profile and everything the parent needs to understand about their child is visible on one screen. Academic standing, strongest skill dimensions, areas needing improvement, predicted outcomes, and specific recommended actions — all in one place, with a one-tap option to generate a PDF summary in Hindi or the parent's preferred language.

7-Dimension Skill Radar

Traditional systems measure one dimension: academic marks. This platform measures seven. Logical-Mathematical intelligence is derived from exam and problem-solving performance. Linguistic intelligence comes from assignment quality, presentation scores, and communication assessments. Spatial-Creative intelligence is measured through design work, project UI quality, and creative task scores. Kinesthetic intelligence reflects lab work quality, hands-on practicals, and sports performance. Interpersonal intelligence is built from group project outcomes, club leadership, and faculty behavioral observations. Intrapersonal intelligence reflects consistency, self-assessment accuracy, and discipline. Technical Depth comes from code quality scores, project complexity, and technology exposure.

Each dimension is scored 0-100 and visualized as a radar chart. A student who is a natural builder shows high Spatial-Creative and Technical Depth with lower Linguistic scores. A student suited for management shows high Interpersonal and Linguistic with moderate Technical. The radar makes these patterns visible instantly to anyone who looks at the profile.

AI Conversation Interface

Every user role has access to an AI conversation interface powered by Claude. This is not a generic chatbot — it is a domain expert that has access to the relevant data for that user's context and answers questions with specificity. A faculty member can ask which topic they should re-teach before the upcoming exam and receive a data-driven answer based on class performance patterns. A dean can ask which students in third year are at placement risk and receive a ranked list with reasoning. A parent can ask what they should do to help their child improve and receive specific, personalized recommendations based on that child's data. A student can ask what they should focus on this month and receive a personalized 30-day action plan.

Early Warning and Alert System

The Alert Agent monitors all student data continuously and generates severity-rated alerts when significant changes or threshold crossings are detected. Alerts have four severity levels: informational, medium, high, and critical. Attendance falling below 75% triggers a high alert sent to faculty and parent simultaneously. A score drop exceeding 20% from a student's personal average triggers a medium alert. Three consecutive missed assignment submissions trigger a high alert. A critical alert is reserved for combinations of factors that historically correlate strongly with exam failure or dropout risk. Every alert includes a recommended intervention action and tracks whether the action was taken and whether it helped.

Potential Gap Report

The Potential Gap Report is the feature that no competing system has. Generated once per semester for every student, it answers one question: given everything we know about this student, if they reached their full potential in their strongest natural dimensions, what would their outcome look like — and what is the precise gap between where they are now and that outcome? The report does not compare students against each other. It compares every student against their own best possible version. This reframes the entire academic conversation from judgment to potential. A student who scores 55% but has untapped project and leadership potential receives a report showing exactly what is possible and how to get there.

Accreditation Report Generator

NBA and NAAC accreditation require extensive documentation: course outcome attainment rates, student performance analytics, faculty workload records, curriculum review data, and outcome-based education metrics. Institutions currently spend weeks manually compiling this data before every accreditation cycle. The Report Generator Agent pulls all required data from the live database and formats it according to the exact specifications required by accreditation bodies. What previously took weeks of administrative work is completed in minutes.

Career Path and Alumni Mirror

The Career Advisor Agent combines a student's 7-dimension skill profile with historical alumni outcome data and live industry demand signals to recommend the top three career paths that match their actual strengths. Each recommendation comes with a confidence percentage, a specific skill gap breakdown, and a learning roadmap. The Alumni Mirror shows students who had a similar skill profile in previous batches and where they are now — real outcomes from real people who graduated from the same institution. This makes career guidance concrete rather than generic.

PART 5

Technology Stack — Detailed Explanation

The technology choices in this project are not arbitrary. Every tool was selected for a specific reason tied to the requirements of the system. This section explains each technology, what it does, and why it is the right choice for this project specifically.

Frontend Layer

Node.js 18+

The runtime environment that makes JavaScript run outside the browser. Next.js cannot start without it. Version 18 introduced stable native fetch API support and significant performance improvements that Next.js 14 depends on. Every developer on the team must have the same Node.js version to ensure consistent behavior across development environments.

Next.js 14

The core frontend framework. Chosen specifically over plain React because of Server Side Rendering. When the dean opens the department dashboard with thousands of data points, SSR pre-loads the data on the server before sending the page to the browser — resulting in fast initial loads regardless of data volume. Next.js also handles routing automatically for all 6 dashboard types, provides API routes for lightweight backend calls, and integrates natively with Vercel for zero-configuration deployment.

TailwindCSS

The styling framework. Building consistent UI across 6 different role dashboards, dozens of components, multiple screen sizes, and multiple institutions requires a systematic design language. Tailwind provides utility classes that keep visual consistency without writing custom CSS for every component. It also produces minimal CSS bundles — critical for dashboard performance in low-bandwidth environments.

Recharts and D3.js

Data visualization libraries. Recharts handles standard charts — line graphs for trend analysis, bar charts for comparison, area charts for attendance patterns. D3.js handles custom visualizations that Recharts cannot produce — the 7-dimension radar chart, the SPI gauge, the skill gap visualization. Both are React-compatible and render cleanly on all screen sizes.

React Query

The data fetching and state management layer. Multiple API calls happen simultaneously when a dashboard loads — student data, alerts, analytics, AI predictions. React Query manages all of these, handles caching so unchanged data is not re-fetched, provides loading states automatically, and retries failed requests. Without React Query, managing this data complexity manually would require thousands of lines of boilerplate code.

NextAuth.js

Authentication and session management. With 5 distinct roles each requiring different data access and different UI, role-based authentication is non-negotiable. NextAuth handles login flows, JWT session management, role verification on every protected route, and the temporary QR-based parent access system. It also supports OAuth providers if the institution wants to integrate with Google or Microsoft accounts.

Node.js Backend Layer

Node.js + Express

The main API server. Every interaction in the platform — marking attendance, loading a dashboard, submitting a project, generating a report — goes through the Express server. It handles request routing, authentication middleware that verifies JWT tokens and checks role permissions on every endpoint, input validation, error handling, and communication with both the database and the Python AI service. Express is chosen for its performance, massive ecosystem, and the ability to share JavaScript types and validation logic with the Next.js frontend.

Prisma ORM

The database query layer. Your PostgreSQL schema has 15+ tables with complex relationships — students to sections, sections to subjects, subjects to faculty, assessments to course outcomes. Prisma generates type-safe query methods so every database interaction is validated at compile time before it runs. It also manages database migrations — when the schema evolves, Prisma generates and tracks the SQL changes needed. Without Prisma, raw SQL queries would need to be written, tested, and maintained manually.

JWT and bcrypt

Security primitives. JWT tokens are issued at login and carry the user's role and institution ID. Every API request includes this token, and the server verifies it cryptographically before processing the request. bcrypt hashes passwords with a salt before storing them, making stored passwords unrecoverable even if the database is compromised. These are non-negotiable for any application handling student data, which is protected by educational privacy regulations.

Multer

File upload middleware. Students submit project files as PDFs or ZIPs. Faculty upload CSV files containing class exam results. Multer receives these files in Express, validates file type and size limits, and passes them to the storage layer. Without Multer, multipart file uploads do not work in Express at all.

Bull Queue with Redis

Background job processing. Several operations in the system are too heavy to run inside a web request — nightly SPI recalculation for all students, bulk WhatsApp notification sending, AI prediction runs for an entire batch, PDF report generation. Bull Queue schedules these as background jobs that run outside the request lifecycle. Users get immediate API responses while heavy work happens asynchronously. Redis stores the job queue state.

Redis

In-memory caching and session storage. Three specific uses: First, caching expensive dashboard calculations so the dean's department overview does not recalculate from scratch on every page load — it reads from cache and refreshes every 5 minutes. Second, storing LangGraph agent checkpoint state between requests so agents can resume from where they left off. Third, powering Bull Queue for background job management.

Python AI Backend Layer

Python 3.11+

The language of the AI layer. LangGraph, LangChain, and the Anthropic Python SDK are all Python-native. The AI backend runs as a separate FastAPI service that the Node.js backend calls when AI work needs to be done. Python 3.11 specifically introduced significant performance improvements — up to 25% faster than 3.10 — which matters when running multiple concurrent agent workflows.

FastAPI

The Python API framework. When Node.js needs AI analysis, it calls FastAPI endpoints. FastAPI is chosen over Flask or Django for three reasons: it is asynchronous by design, enabling multiple agent workflows to run concurrently without blocking; it validates request and response data automatically using Python type hints; and it generates automatic interactive API documentation that the Node.js team can reference when integrating with the AI layer.

LangGraph

The multi-agent orchestration framework — the most important AI dependency in the entire project. LangGraph manages a stateful graph where each node is an AI agent. It defines which agents run in parallel, which wait for others to complete, how state flows between agents, how failures are handled, and how agent memory is persisted across sessions using checkpointing. Without LangGraph, there is no multi-agent system — only disconnected, stateless API calls.

LangChain

The tooling layer for agents. LangGraph agents use LangChain tools to interact with the real world — querying the database, calling external APIs, generating PDFs, dispatching notifications, scoring documents. LangChain provides the abstractions that make these tools easy to define and attach to agents, handles tool execution errors, and provides memory abstractions that work natively with LangGraph.

Anthropic SDK

The official Python library for calling Claude. Every agent in the system uses Claude as its reasoning brain. The SDK handles API authentication, request formatting, response streaming, token counting, error handling, and automatic retry logic. Using the official SDK over raw HTTP requests means benefiting from bug fixes and new feature support automatically as Anthropic releases updates.

Pandas and NumPy

Data processing libraries. Before any data goes into a LangGraph agent, it needs to be cleaned and structured. A student's raw marks across 3 semesters, 6 subjects, and multiple assessment types need to be aggregated into meaningful summaries. Pandas performs this transformation. NumPy handles the mathematical calculations — percentiles for SPI scoring, standard deviations for anomaly detection, correlation matrices for identifying relationships between attendance and performance. Clean structured data is the prerequisite for accurate AI analysis.

SQLAlchemy

Python's database interface. The Python AI agents need to read student data from PostgreSQL to analyze it. SQLAlchemy provides connection pooling — managing multiple simultaneous database connections efficiently — and ORM capabilities so agents can query data using Python objects rather than raw SQL strings.

Infrastructure Layer

PostgreSQL on AWS RDS

The primary database for all application data. PostgreSQL is chosen because the data model has deep relational structure — students, sections, subjects, assessments, skills, predictions all interconnect. AWS RDS manages the PostgreSQL instance with automatic daily backups, multi-availability zone failover, automated security patches, and performance monitoring. For an educational institution where losing student data would be catastrophic, managed database infrastructure is essential.

AWS S3

File storage for project submissions, generated reports, and bulk CSV uploads. S3 provides virtually unlimited storage, 99.999999999% durability guarantee, built-in CDN integration for fast file access, and fine-grained access control. Files uploaded to S3 today will exist reliably in 10 years regardless of application server changes.

AWS EC2

Virtual servers for Node.js and Python backends. Two separate EC2 instances allow independent scaling — if AI workload spikes during exam prediction runs, the Python server scales without affecting the Node.js API server. EC2 provides complete control over the server environment, necessary for a multi-language application with complex dependencies.

Vercel

Frontend hosting built specifically for Next.js. Auto-deploys from GitHub on every push, provides global CDN for fast page loads across India, handles SSL certificates automatically, and enables preview deployments for testing features before merging to production.

Docker

Containerization for consistent environments. Docker packages each service with its exact dependencies so Node.js, Python, and Redis run identically on every developer's machine and on production servers. Without Docker, onboarding a new developer requires hours of environment setup. With Docker, one command starts the complete development environment.

GitHub Actions

Automated CI/CD pipeline. Every code push triggers automated tests, Docker builds, and deployment to staging. Only passing builds deploy to production. This prevents broken code from reaching the platform and automates the repetitive work of manual deployment.

PART 6

LangGraph Multi-Agent Architecture

The AI architecture is the core differentiator of this platform. Instead of making a single API call to Claude and expecting it to do everything, the system uses six specialized agents each expert in a narrow domain, coordinated by LangGraph. This architecture produces significantly more accurate and trustworthy outputs than any single-model approach.

Why Multi-Agent Over Single Model

Dimension	Single Claude Call	LangGraph Multi-Agent
Context quality	One prompt contains everything — context is polluted	Each agent has only the context it needs — clean, focused
Accuracy	Generalizes across all tasks — mediocre at each	Specialized per domain — expert-level at each task
Explainability	Black box output	Each agent returns reasoning alongside results
Failure handling	One failure fails everything	Partial results still useful, other agents continue
Parallelism	Sequential only	Academic and Skill agents run simultaneously
Memory	No memory between calls	LangGraph checkpointing persists state across sessions
Debuggability	Hard to identify where output went wrong	Each agent's output is independently inspectable
Scalability	Cannot split workload	Each agent can be deployed and scaled independently

The Six Agents

Orchestrator Agent	The central coordinator. Receives every incoming request — from faculty, dean, parent, or student — and determines which agents to activate based on the intent of the request. For a parent visit request, it activates Academic Analyst, Skill Profiler, Career Advisor, and Report Generator in the correct sequence. It manages the shared state object that flows through the entire graph, aggregates all agent outputs into a coherent final response, and handles failures gracefully — if one agent fails, it decides whether to retry, skip, or surface a partial result.
Academic Analyst Agent	Specialized in deep academic data analysis. Consumes all assessment data, attendance records, CO mapping, and historical trends for a student or cohort. Produces academic health scores, subject-wise strength and weakness maps, theory versus practical performance gaps, topic-level failure analysis, and semester-over-semester trend direction. Every output includes a reasoning field explaining why the conclusion was reached — enabling explainable AI throughout the system.
Skill Profiler Agent	Specialized in mapping the complete human potential of a student beyond academics. Consumes project submissions, extracurricular records, sports achievements, hackathon results, communication scores, and faculty behavioral observations. Produces the 7-dimension radar chart data, dominant intelligence type classification, hidden strength identification, and soft versus hard skill balance analysis. This agent only activates in parallel with the Academic Analyst — both run simultaneously before the Career Advisor uses their combined outputs.
Career Advisor Agent	Dependent on outputs from both Academic Analyst and Skill Profiler — it cannot run until both complete. Uses the combined academic and skill profile alongside alumni outcome data and industry demand signals to produce career path recommendations, placement readiness scores, 10-year trajectory simulations, alumni mirror matches, and the 30-day personalized action plan. Each career recommendation includes a confidence percentage and the specific reasoning behind the recommendation.

Alert Agent	Runs continuously in the background, monitoring data changes. Every time new attendance or marks data is entered, the Alert Agent checks it against the student's historical baseline and configured thresholds. Generates severity-rated alerts, determines the correct recipient list, avoids duplicate alert spam through deduplication logic, and tracks alert outcomes — whether the intervention was taken and whether it led to improvement. This outcome tracking is how the system learns over time which interventions work.
Report Generator Agent	The final stage of most workflows. Takes outputs from all other agents and formats them for the specific audience and format requested. For a parent visit, it produces a simple visual PDF in the parent's preferred language. For the dean, it produces a detailed strategic report with charts and data tables. For accreditation, it produces the exact format required by NBA or NAAC standards. For WhatsApp digests, it produces concise plain text summaries. One agent, multiple output formats, audience-aware presentation.

The Shared State Object

All agents communicate through a single shared state object managed by LangGraph. This state carries the student ID and requester context at the start, and each agent writes its output to a dedicated field as it completes. When the Career Advisor Agent activates, it reads the academic_analysis and skill_profile fields that the two preceding agents wrote. When the Report Generator activates, it reads everything. This pattern means agents are decoupled from each other — they only interact through state, never directly. If an agent fails, the state records the failure and downstream agents decide how to proceed with partial data.

Agent Memory and Learning

LangGraph supports persistent checkpointing through a PostgreSQL backend. Every agent interaction is saved with its inputs, outputs, and reasoning. When a recommendation is made and the outcome is tracked 30 days later, that outcome feeds back into improving the confidence calibration of future recommendations. Over six months of institutional use, the accuracy of career path recommendations and at-risk detection improves measurably because the system has real outcome data from its own institution. This compounding improvement is what makes the platform more valuable the longer it is used.

PART 7

The Key Decisions That Make It Exceptional

The difference between a college project and an industry-grade product is not the number of features. It is a set of deliberate architectural and product decisions that show deep understanding of the problem. These are the decisions that make this platform stand out.

Prescriptive AI, not just descriptive

Most analytics tools tell you what happened. This platform tells you what to do about it. Every AI output — every prediction, every alert, every recommendation — comes with a specific action. Not 'this student is at risk' but 'this student is at risk because of these specific factors, here is what to do in the next 48 hours, here is what worked for similar cases historically.' This shift from descriptive to prescriptive is what makes the platform actually change behavior rather than just inform it.

Explainable AI as a core requirement

Every prediction the system makes includes its reasoning. The SPI score shows the breakdown of each contributing dimension. The at-risk flag shows which specific data points triggered it. The career recommendation shows why that path was suggested over others. This explainability is not a nice-to-have — it is the reason educational institutions will trust the system. A dean will not act on a prediction they cannot explain. A parent will not accept a recommendation they do not understand. Explainability is what makes AI outputs actionable in high-stakes environments.

One number everyone understands

The Student Potential Index is deliberately designed as a single, trustworthy number that all five user types can immediately understand and act on. This is a critical product decision. Complex multi-dimensional dashboards cause analysis paralysis. A single number with clear meaning — and the ability to drill down to understand it — makes the platform instantly usable by a parent who has never seen it before. Every enterprise product that succeeded at scale — LinkedIn's Social Selling Index, Uber's driver rating, credit scores — used this pattern. One number on the surface, rich analysis underneath.

Event-driven architecture

Data changes in the system emit events. When attendance is marked, an event fires. The Alert Agent service listens for this event and checks thresholds. The Analytics Cache service listens and invalidates stale calculations. The Notification service listens and sends WhatsApp messages if thresholds are crossed. Each service reacts independently. This is how Netflix and Uber are built — services are decoupled, independently scalable, and the failure of one does not cascade to others. It also means adding a new feature that reacts to attendance changes requires only adding a new listener, not modifying existing code.

Multi-tenancy from day one

Building for one college is building a project. Building for one hundred colleges on one platform is building a business. The decision to implement multi-tenancy from the beginning — with institutional configuration, isolated data namespacing, and configurable SPI weights — means the architecture never needs to be rewritten for scale. Every new institution is an additional revenue stream on the same codebase.

The data model is the real product

The software is the entry point. The data is the moat. An institution that uses this platform for three years has three years of structured student intelligence data that exists nowhere else and cannot easily be migrated to a competing system. Every schema decision — storing events not states, maintaining full audit trails, capturing outcome data — is made with this long-term data value in mind. The schema is designed like a data scientist built it, not a developer.

Building for stress moments

Every feature is designed backward from the moment of maximum stress for its user. The parent visit mode is designed for the 10 minutes a parent has with a faculty member. The accreditation report is designed for the night before an inspection. The placement readiness feature is designed for the student with an interview in 48 hours. When software saves someone in their worst moment, they become advocates for it permanently. Designing for these moments requires genuine empathy for users' real experiences.

Feedback loop from day one

Every AI recommendation is logged alongside its outcome. If the Career Advisor recommended a student pursue Product Management and six months later the student's project scores improve significantly after following that path, that is a successful recommendation — logged and used to improve confidence weighting. If the Alert Agent flagged a student as at-risk and after intervention the student recovered, that outcome is tracked. This feedback loop means the system's accuracy improves continuously with institutional use. A system that learns from its own deployment is categorically different from a static rules-based tool.

PART 8

Complete Requirements — Detailed Explanation

Development Environment Requirements

Claude Code

Claude Code is a command-line agentic coding tool that reads and understands your entire codebase simultaneously. For a project of this complexity — three separate services in two languages, 15+ database tables with complex relationships, 6 LangGraph agents with shared state, and multi-role frontend dashboards — Claude Code is not a convenience, it is a practical necessity.

When you change the student schema in PostgreSQL, three files need updating simultaneously: the Prisma schema in the Node.js service, the SQLAlchemy models in Python, and the TypeScript interfaces in Next.js. Claude Code makes all three changes at once because it sees the entire project. When you ask it to build the Academic Analyst Agent, it understands how that agent connects to the state object, the database tools, the FastAPI endpoint that calls it, and the Node.js client that triggers that endpoint. No other coding tool operates at this level of whole-codebase coherence.

Version Control with Git and GitHub

Every line of code must be version-controlled from the first commit. Git provides the ability to roll back any change that breaks the system — critical when LangGraph agent modifications can have unpredictable cascading effects. GitHub provides the remote repository, pull request review process, branch protection rules that prevent direct commits to main, and the integration point for GitHub Actions CI/CD. The repository should be organized as a monorepo — one repository containing the Next.js frontend, Node.js backend, and Python AI service as separate directories with shared configuration.

Environment Variables Management

The application has dozens of sensitive configuration values: database connection strings, the Anthropic API key, AWS credentials, JWT secrets, Redis connection URLs, WhatsApp API tokens. These must never appear in source code. Environment variables are stored in .env files locally, in Vercel's environment settings for the frontend, and in AWS Secrets Manager for production backend services. A single leaked API key can result in significant financial charges or data breaches. Disciplined secrets management is a security requirement, not an optional best practice.

Testing Framework

Three layers of testing are required for a production-grade system. Unit tests verify that individual functions — SPI calculation, alert threshold comparison, PDF generation — produce correct outputs. Jest handles this for Node.js. Pytest handles it for Python. Integration tests verify that the agents, database, and API endpoints work together correctly. End-to-end tests verify complete user flows — a faculty marking attendance, an alert being generated, a notification being sent. Playwright handles E2E testing for the frontend. Without tests, every change to the LangGraph agent graph risks breaking downstream behavior in ways that are only discovered during demos.

Monitoring and Observability

A production system must be observable. When the Alert Agent fails to send a notification, you need to know immediately. When an API endpoint response time increases, you need to see it before users notice. When the database query for the dean's dashboard starts taking 10 seconds instead of 200 milliseconds, you need an alert. AWS CloudWatch provides infrastructure monitoring. Sentry provides application error tracking — capturing and surfacing every exception with the stack trace and user context needed to fix it. These tools are the difference between discovering bugs from user complaints and discovering them before users notice.

PART 9

Data Requirements

The data model is the foundation everything else is built on. Poor data design makes accurate AI analysis impossible. The following categories define every data element the system needs, why it is needed, and what breaks if it is missing.

Identity Data

Students, faculty, dean, parents, and administrators. For students this includes name, roll number, branch, year, section, batch year, email, and phone. For faculty it includes department, subjects teaching, employee ID, and qualification. Parent records map to student records. This data is the root anchor of every other data point. Without clean identity data, AI agents cannot personalize any analysis, reports cannot identify the correct subject, and multi-tenancy breaks because institution boundaries are identity-based.

Academic Performance Data

Every assessment at the individual assessment level — never aggregated before storage. This means unit-wise exam scores per subject, individual assignment scores with submission timestamps, quiz scores, practical lab scores, and project scores with faculty comments. Each assessment maps to one or more Course Outcomes. The granularity is critical: semester-level marks allow the Academic Analyst Agent to say only 'student scored 62%.' Assessment-level marks allow it to say 'student scores 78% in practicals, 71% in assignments, 43% in theory exams, specifically failing normalization in DBMS.' The second is actionable. The first is noise.

Attendance Data

Date-level, subject-level attendance for every student. Not monthly percentages — individual session records. Each record includes date, subject, status, marking faculty, and reason if provided. Monthly percentages can be calculated from day-level data. Day-level data cannot be recovered from monthly percentages. The Alert Agent needs session-level data to detect absence patterns. The audit trail requires session-level records for dispute resolution. Always store the most granular level available.

Skill and Activity Data

Project submissions with tech stack, team composition, file uploads, and AI quality scores. Hackathon and competition participation with level and achievement. Sports records with sport, level, and achievement. Club and society membership with role. Seminars and workshops attended. External certifications completed. Communication assessment scores. This data is what makes the 7-dimension radar chart possible. Without it, the Skill Profiler Agent has nothing to analyze and the platform becomes another marks-based system identical to every ERP the institution already has.

Historical Alumni Data

Anonymized profiles of previous batch graduates with their placement outcomes — company, role, approximate package bracket. This data powers the Alumni Mirror feature and trains the Career Advisor Agent on real institutional outcomes rather than generic industry statistics. Even 30-40 alumni records provide meaningful pattern matching. This is data that only the institution has. No vendor can provide it. Starting collection from day one is essential because it takes time to accumulate and its value compounds with every record added.

Institutional Configuration Data

Branch names and codes, semester structure, subjects per semester per branch, faculty-subject mappings, grading system details, academic calendar, and SPI weight configuration. This layer enables multi-tenancy. When a second institution onboards, they provide this configuration and the entire platform adapts to their structure without code changes. Without this separation between data and configuration, every new institution requires developer intervention.

Audit Trail and Interaction Data

Every data change records who changed what, when, from what previous value. Every AI conversation is stored with the role, query, response, and timestamp. Every alert records when it was generated, when it was seen, what action was taken, and what the outcome was 30 days later. This data serves three purposes: legal compliance for grade and attendance disputes, agent memory for continuing conversations across sessions, and outcome feedback for improving AI accuracy over time. The feedback loop from outcome data is what makes the system a learning platform rather than a static tool.

PART 10

Build Roadmap & Phase Plan

The project is built in six phases. Each phase delivers working, demonstrable value before the next phase begins. This structure ensures that a working product exists at every stage rather than building all components simultaneously and having nothing to show until everything is complete.

Phase 1	Foundation · Weeks 1-3

Monorepo setup with Docker. PostgreSQL schema — all tables, all relationships, Prisma migrations. JWT authentication for all 5 roles. Basic Next.js shell with role-based routing. FastAPI Python service with health check endpoint. GitHub Actions pipeline. Sample data seeding for 50 students across 3 branches.

Phase 2	Core Data Layer · Weeks 4-6

Attendance marking and reporting. Marks entry (individual and bulk CSV). Student profile page with academic data. Faculty dashboard with class overview. Basic SPI calculation engine. Extracurricular and project submission. This phase delivers daily value to faculty before any AI features exist.

Phase 3	AI Intelligence Layer · Weeks 7-10

LangGraph setup with shared state schema. Academic Analyst Agent with database tools. Skill Profiler Agent. Career Advisor Agent dependent on both. Alert Agent with threshold configuration. 7-dimension radar chart in UI. Placement readiness score. AI conversation interface for faculty role.

Phase 4	Dean and Management Layer · Weeks 11-13

Dean department overview dashboard. Faculty performance analytics. Curriculum gap analysis view. Cohort forecasting. Report Generator Agent. NBA/NAAC accreditation report generation. AI conversation interface for dean. Comparative batch analytics.

Phase 5	Parent and Student Experience · Weeks 14-16

Parent visit mode with QR generation. WhatsApp digest integration via Twilio. Student dashboard with full career path features. Alumni mirror. 10-year trajectory simulator. 30-day action plan. Resume builder. Hindi language support for parent-facing screens. Potential Gap Report generation.

Phase 6	Production Hardening · Weeks 17-20

Multi-tenancy configuration layer. Performance optimization — Redis caching for all heavy queries. Load testing. Security audit. Error monitoring with Sentry. Agent memory checkpointing and outcome feedback loop. Documentation. Institutional onboarding flow for second college deployment.

> *The single most important principle across all phases: start small, go deep. Build the student profile and SPI engine so well that those two features alone are impressive. Every feature added on top of a strong foundation compounds in value. Every feature added on top of a weak foundation creates technical debt that eventually makes the whole system fragile. Depth before breadth, always.*

This document covers the complete vision, architecture, technology choices, feature set, data requirements, and build plan for Educator Analytics OS. The project represents a genuine attempt to solve a real problem that affects millions of students in engineering colleges across India — the problem of being reduced to a number on a marksheet when your potential is far richer than any single metric can capture.

Build it well. Build it with integrity. Build it for the students.